9.1.1 The humble beginnings

Where and how did continuous profiling start, and what is the *continuous* in the term referring to? Let's step back a bit. Profiling has been around for decades, with GNU `gprof` (https://ftp.gnu.org/old-gnu/Manuals/gprof-2.9.1/html_mono/gprof.html) and Sun/Solaris DTrace (https://brendangregg.com/dtrace.html) as early examples, further developing in the context of Linux. It was typically considered in the context of performance analysis and often on the (operating) system level.

Now, what is a *profile*, and how does a profile look in practice? We will use an application that emits profiles in the following code. Have a look at this Go code (available in full via ch09/cp/main.go—here, we focus on the relevant bits only) using `pprof` (https://go.dev/blog/pprof):

```
import (
    _ "net/http/pprof"
)

func quickTask() (result string) {
}

func slowTask() (result string) {
}

func main() {
    go func() {
       http.ListenAndServe("localhost:6060", nil)
    }()

    for true {
        result := quickTask()
        fmt.Printf("Quick task result is %s\n", result)
        result = slowTask()
        fmt.Printf("Slow task result is %s\n", result)
    }
}
```

The Go package that automatically collects and exposes profiles for us

A function that relatively quickly completes (10% or less overall CPU time)

A function that relatively is slow (80% or more overall CPU time)

A simple web server exposing the profiling info

Note that the functions `slowTask()` and `quickTask()` don't do anything useful other than using the CPU so that we can see something in the profiler. You can now run the Go program in a terminal session like so:

```
$ go run .
Quick task result is 692530d563c476aea00dd73067d7e146edbece4c17bff4c
Slow task result is 692530d563c476aea00dd73067d7e146edbece4c17bff4c9
...
```

If you navigate to http://localhost:6060/debug/pprof/, you will now see the web server output of the net/http/pprof Go package. Looking at the output here shows you various types of profiles offered, including details on goroutines, heap allocations, and so forth.